

Introduction to Java

Introduction to java:

- It is a simple programming language.
- Java makes writing, compiling, and debugging programming easy.
- It helps to create reusable code and modular programs.
- Java is a class-based, object-oriented programming language and is designed to have as few implementation dependencies as possible.

Features of java

Following are the notable features of Java:

➤ **Object Oriented**

In Java, everything is an Object. Java can be easily extended since it is based on the Object model.

➤ **Platform Independent**

Unlike many other programming languages including C and C++, when Java is compiled, it is not compiled into platform specific machine, rather into platform-independent byte code. This byte code is distributed over the web and interpreted by the Virtual Machine (JVM) on whichever platform it is being run on.

➤ **Simple**

Java is designed to be easy to learn. If you understand the basic concept of OOP Java, it would be easy to master.

➤ **Secure**

With Java's secure feature it enables to develop virus-free, tamper-free systems. Authentication techniques are based on public-key encryption.

➤ **Architecture-neutral**

Java compiler generates an architecture-neutral object file format, which makes the compiled code executable on many processors, with the presence of Java runtime system.

➤ **Portable**

Being architecture-neutral and having no implementation dependent aspects of the specification makes Java portable. The compiler in Java is written in ANSI C with a clean portability boundary, which is a POSIX subset.

➤ **Robust**

Java makes an effort to eliminate error-prone situations by emphasizing mainly on compile time error checking and runtime checking.

➤ Multithreaded

With Java's multithreaded feature it is possible to write programs that can perform many tasks simultaneously. This design feature allows the developers to construct interactive applications that can run smoothly.

➤ Interpreted

Java byte code is translated on the fly to native machine instructions and is not stored anywhere. The development process is more rapid and analytical since the linking is an incremental and light-weight process.

➤ High Performance

With the use of Just-In-Time compilers, Java enables high performance.

➤ Distributed

Java is designed for the distributed environment of the internet.

➤ Dynamic

Java is considered to be more dynamic than C or C++ since it is designed to adapt to an evolving environment. Java programs can carry an extensive amount of run-time information that can be used to verify and resolve accesses to objects at run-time.

Evolution in java

- In 1990 Sun Microsoft Team develop java for Electronics Things but in 1991 java was Comes with the new name Called as OAK then After that in 1992 java was come with the new idea of developing Software's for the Handheld Devices Like Mobile Phones ,Pagers etc.
- But in 1993 Java Was So Popular in the World Wide Web or in the Internet For Developing Games and Graphical Applications with the help of Applets The Most Important usable thing in Graphical Programming.

Keywords:

Keywords are predefined, reserved words used in Java programming that have special meanings to the compiler.

abstract	continue	for	new	switch
assert	default	goto	package	synchronized
boolean	do	if	private	this
break	double	implements	protected	throw
byte	else	import	public	throws
case	enum	Instance of	return	transient
catch	extends	int	short	try
char	final	interface	static	void
class	finally	long	strictfp	volatile
const	float	native	super	while

Primitive Data Types

The eight primitives defined in Java are *int*, *byte*, *short*, *long*, *float*, *double*, *boolean* and *char*. These aren't considered objects and represent raw values.

They're stored directly on the stack

Type	Size (bits)	Minimum	Maximum	Example
<i>byte</i>	8	-2^7	$2^7 - 1$	<i>byte b = 100;</i>
<i>short</i>	16	-2^{15}	$2^{15} - 1$	<i>short s = 30_000;</i>
<i>int</i>	32	-2^{31}	$2^{31} - 1$	<i>int i = 100_000_000;</i>
<i>long</i>	64	-2^{63}	$2^{63} - 1$	<i>long l = 100_000_000_000_000;</i>
<i>float</i>	32	-2^{149}	$(2 \cdot 2^{-23}) \cdot 2^{127}$	<i>float f = 1.456f;</i>
<i>double</i>	64	-2^{1074}	$(2 \cdot 2^{-52}) \cdot 2^{1023}$	<i>double f = 1.456789012345678;</i>
<i>char</i>	16	0	$2^{16} - 1$	<i>char c = 'c';</i>
<i>boolean</i>	1	–	–	<i>boolean b = true;</i>

Java Operators

Operators are used to perform operations on variables and values. Java divides the operators into the following groups:

- Arithmetic Operator,
- Assignment Operator.
- Unary Operator
- Relational Operator,
- Bitwise Operator,
- Logical Operator,
- Ternary Operator
- Increment and Decrement Operator.

Arithmetic Operators

Arithmetic operators are used to perform common mathematical operations.

Operator	Name	Description	Example
+	Addition	Adds together two values	x + y
-	Subtraction	Subtracts one value from another	x - y
*	Multiplication	Multiplies two values	x * y
/	Division	Divides one value by another	x / y
%	Modulus	Returns the division remainder	x % y
++	Increment	Increases the value of a variable by 1	++x
--	Decrement	Decreases the value of a variable by 1	--x

Assignment Operator

Java assignment operator is one of the most common operators. It is used to assign the value on its right to the operand on its left.

Types of Assignment Operators in Java

The Assignment Operator is generally of two types. They are:

1. **Simple Assignment Operator:** The Simple Assignment Operator is used with the “=” sign where the left side consists of the operand and the right side consists of a value. The value of the right side must be of the same data type that has been defined on the left side.
2. **Compound Assignment Operator:** The Compound Operator is used where +, -, *, and / is used along with the = operator.

1. (=) operator:

This is the most straightforward assignment operator, which is used to assign the value on the right to the variable on the left. This is the basic definition of an assignment operator and how it functions.

Syntax:

```
num1 = num2;
```

2. (+=) operator:

This operator is a compound of ‘+’ and ‘=’ operators. It operates by adding the current value of the variable on the left to the value on the right and then assigning the result to the operand on the left.

Syntax:

```
num1 += num2;
```

Unary Operator

The Java unary operators require only one operand. Unary operators are used to perform various operations i.e.:

- incrementing/decrementing a value by one
- negating an expression
- inverting the value of a Boolean

Relational Operator

- Relational operators are used to check the relationship between two operands.
- Here, < operator is the relational operator. It checks if **a** is less than **b** or not.
- It returns either **true** or **false**.

Operator	Description	Example
<code>==</code>	Is Equal To	<code>3 == 5</code> returns false
<code>!=</code>	Not Equal To	<code>3 != 5</code> returns true
<code>></code>	Greater Than	<code>3 > 5</code> returns false
<code><</code>	Less Than	<code>3 < 5</code> returns true
<code>>=</code>	Greater Than or Equal To	<code>3 >= 5</code> returns false
<code><=</code>	Less Than or Equal To	<code>3 <= 5</code> returns true

Bitwise Operators

Bitwise operators in Java are used to perform operations on individual bits.

The various bitwise operators present in Java are:

Operator	Description
<code>~</code>	Bitwise Complement
<code><<</code>	Left Shift
<code>>></code>	Right Shift

>>>	Unsigned Right Shift
&	Bitwise AND
^	Bitwise exclusive OR

Logical Operators

Logical operators are used to check whether an expression is true or false. They are used in decision making.

Operator	Example	Meaning
&& (Logical AND)	expression1 && expression2	true only if both <i>expression1</i> and <i>expression2</i> are true
(Logical OR)	expression1 expression2	true if either <i>expression1</i> or <i>expression2</i> is true
! (Logical NOT)	!expression	true if <i>expression</i> is false and vice versa

Ternary Operator

The ternary operator (conditional operator) is shorthand for the if-then-else statement. Here's how it works.

- If the Expression is true, expression1 is assigned to the variable.
- If the Expression is false, expression2 is assigned to the variable.

Increment and Decrement Operators

- ++
- --
- These are used to increment/decrement a value by 1. Generally to increment a value in the variable x by 1 we use the statement like x=x+1;
- The same statement can be written like x+=1; also using combined assignment operator. Alternatively, we can use x++ or ++x to perform the same thing. Similarly, x=x-1; can be written as --x or x-- also.

Java Variables

- When Java executes a program, the values are stored in containers called variables. It is the name of a memory location.
- It is also a basic unit of storage.
- Variables must be declared before they are used and the changes in variables make actual changes in the memory location.

In Java ,variables are declared as <datatype><variable_name>=<Variable_value>

Types of Variables in Java

Java variables are of 3 types:

1. Java Local Variable
2. Instance Variable in Java
3. Java Static Variable

1. Local Variable in Java

- A local variable is a variable which has value within a particular method or a function. Outside the scope of the function the program has no idea about the variable.
- A local variable cannot be accessed directly outside a function.

Consider a real life **example**, where you have an embarrassing nickname which is known to your parents. But no other person outside your house knows it, right?

That is exactly how a local variable works. It is known locally. Hence the name. Local variables cannot be declared static.

2. Java Instance Variable

- Instance variables are those which are declared inside the body of a class but not within any methods. These differ with each instance of the class created although they have the same identity.
- The variables height and weight are the instance variables, i.e, they are unique for every instance created.
- These cannot be initialized with a static keyword. However one instance of the Person class cannot share the details of the other instance of the same class.

3. Static Variables in Java

- What if we want a variable that is shared across all the instances of a class? That's where static variables come in.
- These are class-level variables which are the same for all the instances generated. As soon as a class loads, the static variables get their initialization.

Static Variables are declared in the following way:

static <variable_name> =<variable_value>

Java Literals

What are Literals in Java?

- Literals have a constant value and a fixed data type and are often known as constants in Java. Literals are assigned to a variable to provide a value to the variable.

Why Use Literals?

- Literal is very useful for implementation in the code as it reduces declaring constant and adding labels by doing it on the same line.

How to use Literals?

- As we saw earlier as well in the discussed topics, a literal is declared along with a data type, a variable name. A literal has an = sign before it which assigns the value of the literal to the variable.

Types of Literals in Java

Generally, there are 5 types of literals that can be further expanded into various other literals. We will take a look at them vividly in this section.

The five main Literal types are:

1. Integer Literal
2. Float/Double Literal
3. Character Literal
4. Boolean Literal
5. String Literal

1.Integer Literal:

Integer literals are basically a number sequence that doesn't contain any decimal point in between them.

There are four types of Integer Literals:

1.1. Decimal Integer:

They are integers having a base value of 10; i.e; containing values between 0 to 9. It can be a positive value(+) or a negative value(-) but it cannot contain any point in between them

1.2. Octal Integer:

They are integers having a base value of 8; i.e; containing values between 0 to 7. All octal numbers must start with a 0.

1.3. Hexadecimal Integer:

They are integers having a base value of 16; i.e; containing values between 0 to 15. It is a special type of integer that contains both digits as well as characters. The digits range from 0 to 9 and the numbers 10 to 15 are replaced by characters a to f. Any integer starting with 0x or 0X is considered to be a hexadecimal integer

1.4. Binary Integer:

They are integers with a base value of 2; i.e; contains only two digits 0 and 1. Binary integers start with a 0b indicating that it is a binary digit

2.Floating Point Literal:

Floating-point literals are values that contain a decimal point in between them. Floating-point literals are generally double data type by default. We can assign them to float data types by adding an f at the end of the value.

3.Boolean Literal:

A Boolean literal is a literal that contains only two values true and false. It is declared using the keyword boolean. It is a very useful literal to declare flag variables in different programs to terminate a looping sequence.

4.Null Literal

Strings in java can also be declared void with a special literal known as null literal. It is basically equivalent to an integer value of 0.

Character Literal:

Character Literals in java are represented with a single quote. The general difference between string literal and character literal is that character literal contains only one character whereas string literal contains a set of characters.

A character literal can be represented in four ways:

1.Single quote character:

Example:

```
char ch = 'A';
```

2. Character Literal as an Integer Literal:

Example:

```
char number = 0065;
```

3. Unicode representation of character Literal:

Example:

```
char uni = '\u0065';
```

4.Escape Sequence:

It is a special kind of character literal which is preceded by a \. Each and every escape sequence has a special feature. Some of the most used escape sequences are:

Escape Sequence	Functionality
\n	Used to insert a new line.
\t	Used to insert a horizontal tab.
\b	Used to insert a blank space.
\v	Used to insert a Vertical tab.

\a	Used to add a small beep sound.
\'	Used to add a single quote inside a string.
\"	Used to add double quotes inside a String.
\\	Used to add a backslash
\r	Used for carriage return.
\?	Used to add a Question mark
\on	Used to represent an octal number
\xHn	Used to represent Hexadecimal number
\uHn	Used to represent Unicode Character Through its hex code.
\0	Null Character
\f	Used to represent formfeed

Few Valid Character Literals:

```
char ch = 'A';
```

```
char ch = '\u0065'
```

```
char ch= 0065
```

```
char ch='\\'
```

```
char ch='%'
```

5.String literals:

There's also a special kind of literal called a class literal, which is formed by taking a type name and appending ".class" to it.

Rules to Use underscore in Literals:

- In the beginning, end or in between the numbers.
- Adjacent to a decimal point in a floating-point literal.
- Prior to an F or L suffix.
- At positions where a string of digits is expected.

Java Control Statements | Control Flow in Java

- Java compiler executes the code from top to bottom. The statements in the code are executed according to the order in which they appear. However, Java

- It provides statements that can be used to control the flow of Java code. Such statements are called control flow statements.
- It is one of the fundamental features of Java, which provides a smooth flow of program.

Java provides three types of control flow statements.

1. Decision Making statements
 - if statements
 - switch statement
2. Loop statements
 - do while loop
 - while loop
 - for loop
 - for-each loop
3. Jump statements
 - break statement
 - continue statement

Decision-Making statements:

As the name suggests, decision-making statements decide which statement to execute and when. Decision-making statements evaluate the Boolean expression and control the program flow depending upon the result of the condition provided. There are two types of decision-making statements in Java, i.e., If statement and switch statement.

1) If Statement:

In Java, the "if" statement is used to evaluate a condition. The control of the program is diverted depending upon the specific condition. The condition of the If statement gives a Boolean value, either true or false. In Java, there are four types of if-statements given below.

1. Simple if statement
2. if-else statement
3. if-else-if ladder
4. Nested if-statement

Let's understand the if-statements one by one.

1.1 Simple if statement:

It is the most basic statement among all control flow statements in Java. It evaluates a Boolean expression and enables the program to enter a block of code if the expression evaluates to true.

Syntax of if statement is given below.

```
if(condition) {  
    statement 1; //executes when condition is true  
}
```

1.2 if-else statement

The if-else statement is an extension to the if-statement, which uses another block of code, i.e., else block. The else block is executed if the condition of the if-block is evaluated as false.

Syntax:

```
if(condition) {  
    statement 1; //executes when condition is true  
}  
else{  
    statement 2; //executes when condition is false  
}
```

1.3 if-else-if ladder:

The if-else-if statement contains the if-statement followed by multiple else-if statements. In other words, we can say that it is the chain of if-else statements that create a decision tree where the program may enter in the block of code where the condition is true. We can also define an else statement at the end of the chain.

Syntax of if-else-if statement is given below.

```
if(condition 1) {  
    statement 1; //executes when condition 1 is true  
}  
else if(condition 2) {  
    statement 2; //executes when condition 2 is true  
}  
else {  
    statement 2; //executes when all the conditions are false  
}
```

1.4 Nested if-statement

In nested if-statements, the if statement can contain a **if** or **if-else** statement inside another if or else-if statement.

Syntax of Nested if-statement is given below.

```
if(condition 1) {  
    statement 1; //executes when condition 1 is true  
    if(condition 2) {  
        statement 2; //executes when condition 2 is true  
    }  
}
```

```

}
else{
statement 2; //executes when condition 2 is false
}
}

```

Switch Statement:

- In Java, Switch statements are similar to if-else-if statements. The switch statement contains multiple blocks of code called cases and a single case is executed based on the variable which is being switched.
- The switch statement is easier to use instead of if-else-if statements. It also enhances the readability of the program.

Points to be noted about switch statement:

- The case variables can be int, short, byte, char, or enumeration. String type is also supported since version 7 of Java
- Cases cannot be duplicate
- Default statement is executed when any of the case doesn't match the value of expression. It is optional.
- Break statement terminates the switch block when the condition is satisfied. It is optional, if not used, next case is executed.
- While using switch statements, we must notice that the case expression will be of the same type as the variable. However, it will also be a constant value.

The **syntax** to use the switch statement is given below.

```

switch (expression){

case value1:

statement1;
break;
.
.
.
case valueN:
statementN;
break;
default:
default statement;
}

```

Loop Statements

- In programming, sometimes we need to execute the block of code repeatedly while some condition evaluates to true. However, loop statements are used to

execute the set of instructions in a repeated order. The execution of the set of instructions depends upon a particular condition.

➤ In Java, we have three types of loops that execute similarly. However, there are differences in their syntax and condition checking time.

1. for loop
2. while loop
3. do-while loop

Let's understand the loop statements one by one.

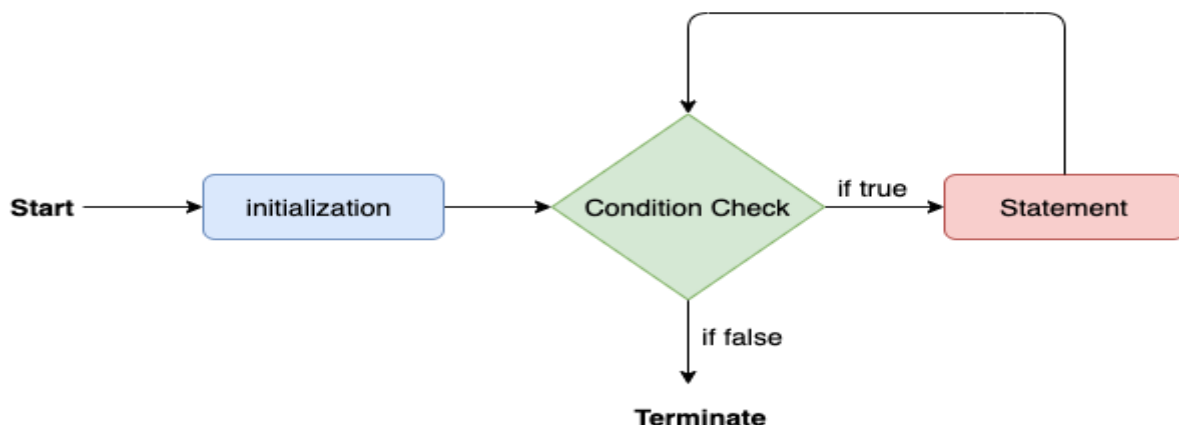
Java for loop

- In Java, for loop is similar to C and C++
- It enables us to initialize the loop variable, check the condition, and increment/decrement in a single line of code. We use the for loop only when we exactly know the number of times, we want to execute the block of code.

Syntax

```
for(initialization, condition, increment/decrement) {  
    //block of statements  }  
}
```

The flow chart for the for-loop is given below.



Java while loop

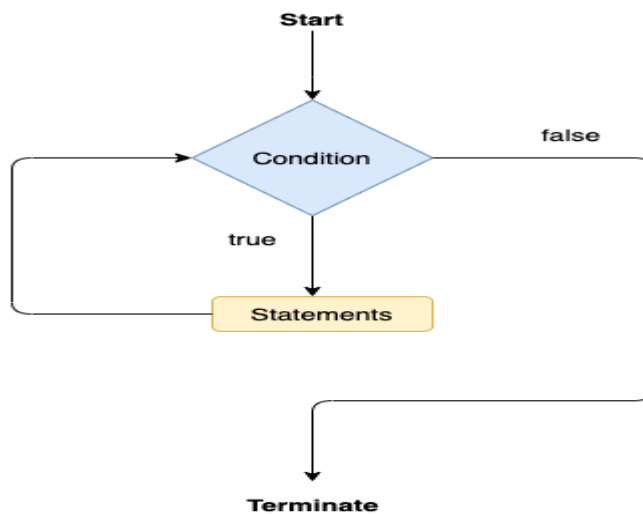
- It also used to iterate over the number of statements multiple times. However, if we don't know the number of iterations in advance, it is recommended to use a while loop.
- Unlike for loop, the initialization and increment/decrement doesn't take place inside the loop statement in while loop.

- It is also known as the entry-controlled loop since the condition is checked at the start of the loop.
- If the condition is true, then the loop body will be executed; otherwise, the statements after the loop will be executed.

The **syntax** of the while loop is given below.

```
while(condition){  
  //looping statements  
}
```

The flow chart for the while loop is given in the following image.



Java do-while loop

The do-while loop

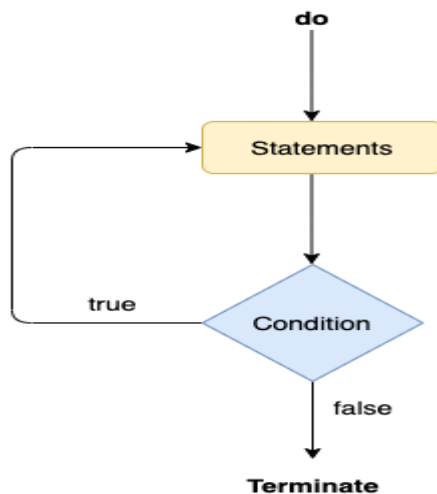
Checks the condition at the end of the loop after executing the loop statements. When the number of iteration is not known and we have to execute the loop at least once, we can use do-while loop.

It is also known as the exit-controlled loop since the condition is not checked in advance.

The **syntax** of the do-while loop is given below.

```
do  
{  
  //statements  
} while (condition);
```

The flow chart of the do-while loop is given in the following image.



Jump Statements

- Jump statements are used to transfer the control of the program to the specific statements. In other words, jump statements transfer the execution control to the other part of the program. There are two types of jump statements in Java, i.e., break and continue.

Java break statement

- As the name suggests, the break statement
- is used to break the current flow of the program and transfer the control to the next statement outside a loop or switch statement. However, it breaks only the inner loop in the case of the nested loop.
- The break statement cannot be used independently in the Java program, i.e., it can only be written inside the loop or switch statement.

Classes and Objects

Classes in Java:

A class is a logical entity or blueprint from which objects can be created. A class can be described as a group of objects having common properties.

Classes in java contain:

- Fields,
- Methods,
- Constructors,
- Blocks,
- Nested Classes,
- Interfaces.

Syntax for class

```
class <class name>{  
    //Statements  
}
```

Objects in Java:

An object is a real-life entity of a class. A class is a logical entity whereas an object is a physical entity. A class is defined as a template or blueprint, it is only when an object is declared that memory is assigned to the program. An object has a proper state and behaviour. Objects are also known as instances of a class.

Syntax of an object:

<Class name> <Object name>= new <class name>();

Java package

- A **java package** is a group of similar types of classes, interfaces and sub-packages.
- Package in java can be categorized in two form,
 - built-in package
 - user-defined package.
- There are many built-in packages such as java, lang, awt, javax, swing, net, io, util, sql etc.
- Here, we will have the detailed learning of creating and using user-defined packages.

Creating Packages in Java:

Creating packages in Java is very simple. Just include the package command and the name of the package after it. Remember that this should be the first line of your program.

Advantage of Java Package

- 1) Java package is used to categorize the classes and interfaces so that they can be easily maintained.
- 2) Java package provides access protection.
- 3) Java package removes naming collision

How to compile java package

If you are not using any IDE, you need to follow the **syntax** given below:

javac -d directory javafilename

How to run java package program

You need to use fully qualified name e.g. mypack.Simple etc to run the class.

To Compile: javac -d . Simple.java

To Run: java mypack.Simple

Output: Welcome to package

How to access package from another package?

There are three ways to access the package from outside the package.

1. import package.*;
2. import package.classname;
3. fully qualified name.

1) Using packagename.*

- If you use package.* then all the classes and interfaces of this package will be accessible but not subpackages.
- The import keyword is used to make the classes and interface of another package accessible to the current package.

2) Using packagename.classname

- If you import package.classname then only declared class of this package will be accessible.

3) Using fully qualified name

- If you use fully qualified name then only declared class of this package will be accessible. Now there is no need to import. But you need to use fully qualified name every time when you are accessing the class or interface.
- It is generally used when two packages have same class name

e.g. java.util and java.sql packages contain Date class.

Access Specifiers or modifiers:

- **There is no difference between access specifier and access modifier in Java.** They both mean the same.
- There are two types of modifiers in Java:
 - **access modifiers**
 - **non-access modifiers.**
- The access modifiers in Java specifies the accessibility or scope of a field, method, constructor, or class. We can change the access level of fields, constructors, methods, and class by applying the access modifier on it.

There are four types of Java access modifiers:

1. **Private:** The access level of a private modifier is only within the class. It cannot be accessed from outside the class.

2. **Default:** The access level of a default modifier is only within the package. It cannot be accessed from outside the package. If you do not specify any access level, it will be the default.
3. **Protected:** The access level of a protected modifier is within the package and outside the package through child class. If you do not make the child class, it cannot be accessed from outside the package.
4. **Public:** The access level of a public modifier is everywhere. It can be accessed from within the class, outside the class, within the package and outside the package.

Understanding Java Access Modifiers

Let's understand the access modifiers in Java by a simple table.

Access Modifier	within class	within package	outside package by subclass only	outside package
Private	Y	N	N	N
Default	Y	Y	N	N
Protected	Y	Y	Y	N
Public	Y	Y	Y	Y

1) Private

The private access modifier is accessible only within the class.

Role of Private Constructor

If you make any class constructor private, you cannot create the instance of that class from outside the class.

2) Default

If you don't use any modifier, it is treated as **default** by default. The default modifier is accessible only within package. It cannot be accessed from outside the package. It provides more accessibility than private. But, it is more restrictive than protected, and public.

3) Protected

The **protected access modifier** is accessible within package and outside the package but through inheritance only.

The protected access modifier can be applied on the data member, method and constructor. It can't be applied on the class.

It provides more accessibility than the default modifier.

4) Public

The **public access modifier** is accessible everywhere. It has the widest scope among all other modifiers.

Constructors - Default and Parameterized:

Constructors in Java

- In Java, a constructor is a block of codes similar to the method. It is called when an instance of the class is created.
- At the time of calling constructor, memory for the object is allocated in the memory.
- It is a special type of method which is used to initialize the object.
- Every time an object is created using the new() keyword, at least one constructor is called. It calls a default constructor if there is no constructor available in the class. In such case, Java compiler provides a **default constructor by default**.

Rules for creating Java constructor

There are two rules defined for the constructor.

1. Constructor name must be the same as its class name
2. A Constructor must have no explicit return type
3. A Java constructor cannot be abstract, static, final, and synchronized

Types of Java constructors

There are two types of constructors in Java:

1. **Default constructor (no-arg constructor)**
2. **Parameterized constructor**

Java Default Constructor

- A constructor is called "Default Constructor" when it doesn't have any parameter.

Syntax of default constructor:

```
<class_name>(){}
```

What is the purpose of a default constructor?

- The default constructor is used to provide the default values to the object like 0, null, etc., depending on the type.

Java Parameterized Constructor

- A constructor which has a specific number of parameters is called a parameterized constructor.

Why use the parameterized constructor?

- The parameterized constructor is used to provide different values to distinct objects. However, you can provide the same values also.

Java Copy Constructor

- There is no copy constructor in Java. However, we can copy the values from one object to another like copy constructor in C++.
- There are many ways to copy the values of one object into another in Java. They are:
 - By constructor
 - By assigning the values of one object into another
 - By clone() method of Object class.

Reference in java:

- The **“this”** is a keyword in Java which is used as a reference to the object of the current class, within an instance method or a constructor.
- Using this you can refer the members of a class such as constructors, variables and methods.
- Using **“this”** you can –
 - Differentiate the instance variables from local variables if they have same names, within a constructor or a method.

Java static keyword:

The **static keyword** in Java is used for memory management mainly. We can apply static keyword with variables, methods, blocks and nested classes. The static keyword belongs to the class than an instance of the class.

The static can be:

1. Variable (also known as a class variable)
2. Method (also known as a class method)
3. Block
4. Nested class

1) Java static variable

If you declare any variable as static, it is known as a static variable.

- The static variable can be used to refer to the common property of all objects (which is not unique for each object), for example, the company name of employees, college name of students, etc.
- The static variable gets memory only once in the class area at the time of class loading.

Advantages of static variable:

It makes your program **memory efficient** (i.e., it saves memory).

2) Java static method

If you apply static keyword with any method, it is known as static method.

- A static method belongs to the class rather than the object of a class.
- A static method can be invoked without the need for creating an instance of a class.
- A static method can access static data member and can change the value of it.

Restrictions for the static method:

There are two main restrictions for the static method. They are:

1. The static method can not use non static data member or call non-static method directly.
2. this and super cannot be used in static context.

3) Java static block

- Is used to initialize the static data member.
- It is executed before the main method at the time of classloading.

4) Java Static Nested Class

- Java allows us to define a class but not a static class which simply means that we can define a class static only if it is a nested class.
- In Java, a class can be static when it is a nested class. Nested static class doesn't need any reference of the Outer class.
- Non-static members are not accessed by a static class.

Exploring Basic Java Class Libraries in java

Object class :

- The **Object class** is the parent class of all the classes in java by default. In other words, it is the topmost class of java.
- The Object class is beneficial if you want to refer any object whose type you don't know. Notice that parent class reference variable can refer the child class object, know as upcasting.

Various techniques of object class creation in java:

There are a total of five different methods using which we can instantiate an object. They are as follows:

- Using new keyword
- Using newInstance() method of Class class
- Using newInstance() method of constructor class
- Using clone() method

- Using deserialization

Let us understand each of these methods individually.

1. Using the new keyword:

The new keyword is the most common technique used to instantiate an object. The new keyword calls the constructor of the object and creates a new instance of the class.

Syntax:

ClassName **ObjectName** = **new** **Class Name**();

2. Using newInstance() Method of Class class:

The newInstance() method of the Class class calls the default constructor to create the object. The method returns the newly created instance of the class of which the object is a part. The newInstance() method is part of the Constructor class.

Syntax:

public T newInstance() throws InstantiationException, IllegalAccessException

The method throws the following exceptions:

- IllegalAccessException
- InstantiationException
- ExceptionInInitializerError

We can create an object in the following ways:

- **ClassName** **object** = **ClassName.class.newInstance()**;
- **ClassName** **object** = **(ClassName) Class.forName("Name of Full Class").newInstance()**;

The forName() method is a static method which takes className as a parameter and returns the object of the class. It just loads the class to the program but does not create an object. If the class cannot be loaded it gives a ClassNotFoundException and a LinkageError if the linkage has failed.

The newInstance() method only works if the class has a public default constructor.

3. Using newInstance() Method of Constructor class:

This method is similar to the previous method. It is also known as a reflective way to create an object. The method is defined in the Constructor class of java.lang.reflect package. Unlike the previous method, here we can use parameterized constructors as well as private constructors. This is the reason why this method is preferred over the Class class newInstance() method.

Syntax:

Public T newInstance(Object... initargs) throws InstantiationException, IllegalAccessException, IllegalArgumentException, InvocationTargetException

The newInstance() method passes an array of objects as an argument. It returns a newly created object by calling the constructor of the class.

The method throws the following exceptions:

- IllegalAccessException
- IllegalArgumentException
- InstantiationException
- InvocationTargetException
- ExceptionInInitializerError

Syntax of creating object:

```
Constructor<ClassName> constructor = ClassName.class.getConstructor();  
ClassName ObjectName = constructor.newInstance();
```

4. Using the clone() Method:

The clone() method is present inside the Object class. It can create a copy of a preexisting object and return it. It copies the previous object's properties into the newly created object. The clone() method doesn't call the constructor. If the object to be copied does not support a cloneable interface, the method will throw CloneNotSupportedException.

Syntax:

protected Object clone() throws CloneNotSupportedException

Creating a new object from old object:

```
ClassName newobjectname = (ClassName) oldobjectname.clone();
```

5. Using Deserialization:

Serialization in java means, to convert an object into a sequence of byte-stream. The reverse of serialization is deserialization. The JVM creates an object whenever we deserialize an object. It does not use a constructor to create objects. To use deserialization, the serialization interface must be implemented at first.

Syntax:

Serialization:

```
public final void writeObject(object x) throws IOException
```

Deserialization:

```
public final Object readObject() throws IOException, ClassNotFoundException.
```

Wrapper Class in Java

- Wrapper classes, as the name suggests, wraps around or encapsulates primitive datatypes in Java. We talked about this in one of our previous articles so be sure to check them out too.

- Wrapper classes, simply put, is basically a class for converting a primitive datatype, to an object for specific functions.
- This is useful because primitive datatypes are generally immutable. Also, in Java, everything is object-oriented in nature. So if you want to have a deeper understanding of how this work, read along.

Primitive Datatypes	Wrapper Classes
byte	Byte
boolean	Boolean
char	Character
int	Integer
long	Long
short	Short
double	Double
float	Float

Type casting in java:

- In Java, **type casting** is a method or process that converts a data type into another data type in both ways manually and automatically.
- The automatic conversion is done by the compiler and manual conversion performed by the programmer.
- To Convert a value from one data type to another data type is known as **type casting**.

Types of Type Casting

There are two types of type casting:

- Widening Type Casting
- Narrowing Type Casting

Widening Type Casting

- It Converting a lower data type into a higher one is called **widening** type casting.
- It is also known as **implicit conversion** or **casting down**. It is done automatically.
- It is safe because there is no chance to lose data. It takes place when:
 - Both data types must be compatible with each other.
 - The target type must be larger than the source type.

Narrowing Type Casting

- It Converting a higher data type into a lower one is called **narrowing** type casting.
- It is also known as **explicit conversion** or **casting up**.
- It is done manually by the programmer. If we do not perform casting then the compiler reports a compile-time error.

Using scanner class in java:

- Scanner class in Java is found in the java.util package. Java provides various ways to read input from the keyboard, the java.util.Scanner class is one of them.
- The Java Scanner class breaks the input into tokens using a delimiter which is whitespace by default. It provides many methods to read and parse various primitive values.
- The Java Scanner class is widely used to parse text for strings and primitive types using a regular expression. It is the simplest way to get input in Java. By the help of Scanner in Java, we can get input from the user in primitive types such as int, long, double, byte, float, short, etc.
- The Java Scanner class extends Object class and implements Iterator and Closeable interfaces.
- The Java Scanner class provides nextXXX() methods to return the type of value such as nextInt(), nextByte(), nextShort(), next(), nextLine(), nextDouble(), nextFloat(), nextBoolean(), etc. To get a single character from the scanner, you can call next().charAt(0) method which returns a single character.

Java Scanner Class Declaration

```
public final class Scanner
    extends Object
    implements Iterator<String>
```

How to get Java Scanner

- To get the instance of Java Scanner which reads input from the user, we need to pass the input stream (System.in) in the constructor of Scanner class.

String handling in java:

- Strings, which are widely used in Java programming, are a sequence of characters. In Java programming language, strings are treated as objects.
- The Java platform provides the String class to create and manipulate strings.
- The most direct way to create a string is to write –
String greeting = "Hello world!";
- Whenever it encounters a string literal in your code, the compiler creates a String object with its value in this case, "Hello world!".

Java Date and Time APIs:

- Java provide the date and time functionality with the help of two packages java.time and java.util.

- The package `java.time` is introduced in Java 8, and the newly introduced classes tries to overcome the shortcomings of the legacy `java.util.Date` and `java.util.Calendar` classes.

Classical Date Time API Classes

The primary classes before Java 8 release were:

- **Java.lang.System:** The class provides the `currentTimeMillis()` method that returns the current time in milliseconds. It shows the current date and time in milliseconds from January 1st 1970.
- **java.util.Date:** It is used to show specific instant of time, with unit of millisecond.
- **java.util.Calendar:** It is an abstract class that provides methods for converting between instances and manipulating the calendar fields in different ways.
- **java.text.SimpleDateFormat:** It is a class that is used to format and parse the dates in a predefined manner or user defined pattern.